Flat Combining for Concurrent Queues in OCaml

Design, Correctness, and Performance Evaluation

Chaitanya Sai Teja
cs22b036@smail.iitm.ac.in

Yashwanth Sai
cs22b002@smail.iitm.ac.in

Rushi
cs22b040@smail.iitm.ac.in

May 1, 2026

Abstract

We present a flat-combining queue implemented in OCaml and evaluated against two baselines: a direct mutex-protected queue and a blocking queue using condition variables. The project studies whether batching many pending queue operations through one combiner can reduce synchronization overhead under contention. The implementation consists of a generic flat-combining wrapper, a queue instantiation over a sequential FIFO specification, correctness tests, and a benchmark matrix over enqueue-heavy, balanced, and dequeue-heavy producer/consumer workloads. The main finding is conditional: flat combining is slower than a mutex queue at low contention, but becomes competitive when enough requests accumulate for the combiner to process larger batches. In the extended benchmark, flat combining overtakes the mutex queue for the dequeue-heavy workload at 12 threads and for the balanced workload at 24 threads, while no crossover is observed for the enqueue-heavy workload up to 32 threads.

1 Goals

The project goal is to implement and evaluate a concurrent data structure that is relevant to shared-memory concurrency but not a direct reproduction of a standard class implementation. The chosen topic is flat combining for queues. A queue is a useful target because its sequential semantics are simple, while its concurrent implementation exposes important issues: synchronization overhead, contention, fairness, progress, and linearizability.

The central research question is:

> When does a flat-combining queue become competitive with, or faster than, a simple mutex-protected queue?

The question is motivated by the fact that queue operations are small. In a concurrent queue, the cost of synchronization can dominate the useful work of enqueueing or dequeueing one value. A mutex queue serializes every operation by locking the shared queue. Flat combining also serializes access to the underlying sequential queue, but it changes who pays the synchronization cost: callers publish operations into slots, and one combiner applies many pending operations in a batch.

The project therefore has four concrete goals. First, implement a usable concurrent queue API with enqueue and dequeue operations. Second, implement the flat-combining mechanism generically over a sequential object specification, so the wrapper is not hard-coded only for queues. Third, test the implementation using both deterministic tests and linearizability testing. Fourth, evaluate performance across multiple thread counts and producer/consumer mixes, comparing flat combining with mutex and blocking queue baselines.

2 Background

A sequential FIFO queue supports enqueue and dequeue. The concurrent version must preserve a sequential explanation of operations even when many domains invoke them at the same time. The standard correctness condition for this is linearizability: every operation should appear to take effect at one instant between its call and return [2].

The simplest concurrent queue is a sequential queue protected by one mutex. This design is easy to understand because every operation has a clear critical section. Its weakness is that all domains contend for the same lock. Under low contention, that is often acceptable. Under high contention, lock handoff, cache-line movement, and repeated synchronization can become more expensive than the queue operation itself.

Flat combining, introduced by Hendler, Incze, Shavit, and Tzafrir [1], uses a different synchronization strategy. Threads do not all mutate the data structure directly. Instead, they publish operation requests in a shared announcement structure. One thread becomes the combiner, scans the announced requests, applies them to a sequential object, and writes back the results. This design deliberately trades away parallel updates to the data structure in exchange for lower synchronization overhead and batching.

This trade-off is the heart of the project. If each combiner pass handles only one operation, flat combining is likely to lose because it pays for publication slots, combiner election, scanning, and result collection. If many operations are pending together, the combiner can amortize that overhead across a batch. The expected performance is therefore workload-dependent rather than universally better.

The implementation uses OCaml 5 domains for parallel execution. It uses atomic references and compare-and-set operations for slot state transitions and combiner election, domain-local storage for stable per-domain publication slots, a mutex only for slot allocation, and condition variables in the blocking baseline.

3 Tasks Undertaken

Sequential specification. `lib/types.ml` defines the sequential queue used by the flat-combining wrapper. The state is an `int Queue.t`. The operation type is either `Enq of int` or `Deq`; the result type is `int option`. Enqueue inserts a value and returns `None`. Dequeue returns `Some x` when a value is available and `None` when the queue is empty. This module is the sequential contract that the concurrent wrapper must preserve.

Flat-combining wrapper. The main implementation is in `lib/fc_wrapper.ml`. It defines a functor over a `SEQUENTIAL` module with an initial state and an `apply` function. Each domain publishes a request in a slot whose atomic state is one of three values: `Empty`, `Pending op`, or `Completed result`. A caller changes its slot from `Empty` to `Pending`; a combiner applies the operation and changes the slot to `Completed`; the caller reads the result and resets the slot to `Empty`.

Combiner election is controlled by an atomic Boolean flag. A waiting caller tries to compare-and-set this flag from `false` to `true`. If it succeeds, it scans all slots, applies every pending operation to the sequential state, fulfills those slots, and continues scanning until no progress is made. If it fails, another domain is already combining, so the caller relaxes the CPU and waits for its own slot to become completed.

The linearization point of an operation is the call to `Q.apply` inside the combiner. At that point, the operation is applied to the unique sequential queue state and its result is determined. This is why the generic wrapper can be reasoned about separately from the queue-specific API: the wrapper serializes requests, while the supplied sequential specification defines the object behavior.

Queue API and baselines. The file `lib/fc_queue.ml` instantiates the wrapper with the sequential queue specification and exposes `create`, `enqueue`, `dequeue`, and statistics functions. The default queue has eight publication slots, while the benchmark creates queues with `threads + 1` slots to avoid slot exhaustion.

The project also implements two baselines. `lib/mutex_queue.ml` protects an OCaml queue with a single mutex and provides non-blocking dequeue. `lib/blocking_queue.ml` implements a bounded circular-buffer queue with a mutex and two condition variables, `not_empty` and `not_full`. The mutex queue is the most direct comparison because it has the same high-level non-blocking dequeue behavior as the flat-combining queue. The blocking queue is included as a practical producer/consumer baseline.

Testing and verification. The manual tests in `test/manual_tests.ml` check three scenarios. The sequential test enqueues 1, 2, and 3, then verifies FIFO dequeue order and empty-queue behavior. The concurrent enqueue test spawns four domains, each inserting 25 values, then drains 100 values and checks that no values were lost. The mixed test runs one producer and one consumer concurrently until 100 values have been consumed.

The stronger correctness test is `test/qcheck_tests.ml`, which uses QCheck-Lin domain tests. It checks whether generated concurrent histories for enqueue and dequeue can be explained by some valid sequential queue history. Empty dequeues are encoded with `min_int`, while generated enqueue values are small naturals, so the sentinel does not collide with real values. The test count is 500 histories. This directly targets linearizability, the key correctness property for a concurrent queue.

Benchmark harness. The benchmark driver constructs producer/consumer workloads from a thread count, operations per thread, and enqueue percentage. It measures wall-clock time with `Unix.gettimeofday`, computes throughput as operations per second, and reports the flat-combining average batch size:

$$\text{avg batch size} = \frac{\text{combined operations}}{\text{combine cycles}}.$$

The evaluation uses the extended 2–32 thread benchmark data and compares it with a second 2–8 thread run where relevant. Since the two runs have different 2–8 values, they are treated as separate measurements rather than merged. The available graph and raw-data files are not used because they do not contain complete plot data.

4 Evaluation

The extended benchmark uses thread counts $\{2, 4, 6, 8, 12, 16, 24, 32\}$, `ops_per_thread = 5000`, and three mixes: enqueue-heavy 80/20, balanced 50/50, and dequeue-heavy 20/80. For dequeue-heavy workloads, the benchmark pre-fills the queue so consumers can complete the target number of successful dequeue operations.

The table shows that flat combining is poor at low thread counts. At 2 threads, the average batch size is close to 1 in all workloads, so the combiner is not amortizing its overhead. The mutex queue is much faster in this regime because it takes a short direct critical section and avoids publication-slot scanning.

As contention increases, the batch size rises. In the enqueue-heavy workload, it grows from 1.034 at 2 threads to 9.169 at 32 threads. The balanced and dequeue-heavy workloads show the same overall effect. This confirms that the implementation is doing the expected batching work.

The crossover is workload-dependent. In the dequeue-heavy 20/80 workload, flat combining first beats the mutex baseline at 12 threads: 275,676 ops/s compared with 250,995 ops/s. It is also ahead at 16 and 32 threads. In the balanced 50/50 workload, flat combining first beats the

Table 1: Primary extended benchmark summary. Throughput is operations per second.

Thr.	Mix	FC	Mutex	Blocking	Batch	FC vs Mutex
2	80/20	64,868	315,039	282,869	1.034	-79.5%
2	50/50	82,827	322,947	230,155	1.068	-74.4%
2	20/80	172,304	362,199	336,938	1.016	-52.4%
4	80/20	217,332	250,363	188,952	1.998	-13.2%
4	50/50	183,357	307,968	236,833	1.860	-40.5%
4	20/80	190,674	269,023	208,333	2.375	-29.1%
6	80/20	171,164	224,242	189,707	3.055	-23.7%
6	50/50	168,880	289,037	221,157	2.269	-41.6%
6	20/80	228,003	273,210	206,130	2.688	-16.6%
8	80/20	176,896	217,978	181,181	4.797	-18.9%
8	50/50	162,713	256,698	204,859	3.089	-36.6%
8	20/80	203,965	234,534	158,273	2.133	-13.0%
12	80/20	168,367	184,002	182,239	6.585	-8.5%
12	50/50	222,044	251,314	197,563	7.476	-11.7%
12	20/80	275,676	250,995	184,079	5.957	+9.8%
16	80/20	179,977	183,017	171,010	8.065	-1.7%
16	50/50	195,350	217,651	181,965	7.581	-10.3%
16	20/80	234,209	220,047	187,547	8.140	+6.4%
24	80/20	161,527	191,760	163,486	9.051	-15.8%
24	50/50	207,865	193,473	168,848	9.007	+7.4%
24	20/80	199,081	201,182	157,005	8.545	-1.0%
32	80/20	143,358	169,659	151,336	9.169	-15.5%
32	50/50	173,783	196,224	169,264	9.394	-11.4%
32	20/80	182,203	166,735	154,632	9.425	+9.3%

mutex queue at 24 threads: 207,865 ops/s compared with 193,473 ops/s. In the enqueue-heavy 80/20 workload, there is no crossover up to 32 threads; the closest point is 16 threads, where flat combining is only 1.7% behind the mutex queue.

The blocking queue is useful as a second baseline, but it follows a different synchronization model. It blocks on condition variables when empty or full, while the flat-combining and mutex queues use non-blocking dequeue and consumer retry loops. It is competitive in some low-thread cases but does not benefit from batching.

A second 2–8 thread benchmark run gives different values from the 2–8 subset of the extended benchmark. For example, in the balanced 2-thread case it reports 236,172 ops/s for flat combining, while the extended benchmark records 82,827 ops/s. Since the measurement records do not include enough metadata to identify the cause, the second run is treated as a separate measurement and not averaged with the extended results. This discrepancy is a limitation of the benchmark data.

5 Reflection on the Use of LLMs

We used Gemini, GitHub Copilot, and ChatGPT/Codex as assistants during the project. Gemini was useful for understanding the project statement, learning background concepts, and generating starter code ideas. GitHub Copilot helped speed up routine coding by suggesting small code fragments and boilerplate while implementing and editing files. ChatGPT/Codex was useful for understanding the project structure, organizing the report, and improving explanations of the flat-combining algorithm. Together, the tools helped connect the implementation files to the required report sections: the sequential specification, wrapper, queue API, baselines, tests, and benchmark results.

The LLMs were not treated as a substitute for understanding the code. Generated suggestions and starter code had to be checked against the implementation, especially the slot protocol, combiner election, linearization point, and benchmark data. One important caution was that two benchmark runs gave different 2–8 thread values, so the results were described without merging them.

Overall, the LLMs helped with learning, faster coding, writing, structure, and clarity, but the final reasoning and responsibility for correctness remained with us.

6 Conclusions

The project implements a generic flat-combining wrapper, a flat-combining queue, two baselines, manual tests, linearizability tests, and a benchmark harness. The design cleanly separates the sequential queue specification from the concurrent wrapper. The wrapper handles publication slots, combiner election, batching, result delivery, and statistics, while the queue module supplies the sequential FIFO behavior.

The answer to the research question is nuanced. Flat combining is not a universal improvement over a mutex queue. At low contention, it is slower because the overhead of publication, scanning, and result handoff is not amortized. At higher contention, it can become faster because the combiner processes larger batches. In the extended benchmark, flat combining crosses over in the dequeue-heavy workload at 12 threads and in the balanced workload at 24 threads, but not in the enqueue-heavy workload up to 32 threads.

The main limitation of the project is evaluation depth. The benchmark results show clear trends, but the evaluation does not include repeated trials, confidence intervals, hardware details, or a generated graph from a complete raw-data file. Future work includes repeated measurements with variance, comparison against a lock-free queue, and adaptive combiner policies such as bounded combining rounds or backoff.

7 Contributions

The contribution split is:

Member	**Contribution**	**Main responsibilities**
Chaitanya (cs22b036)	40%	flat-combining implementation and presentation
Yashwanth (cs22b002)	30%	benchmarks, report and result collection
Rushi (cs22b040)	30%	correctness tests and experimental setup
Total	**100%**	

References

[1] Danny Hendler, Itai Incze, Nir Shavit, and Moran Tzafrir. Flat combining and the synchronization-parallelism tradeoff. In *Proceedings of the Twenty-Second Annual ACM Symposium on Parallelism in Algorithms and Architectures*, SPAA '10, pages 355–364. Association for Computing Machinery, 2010. doi:10.1145/1810479.1810540.

[2] Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. *The Art of Multiprocessor Programming.* Morgan Kaufmann, 2nd edition, 2020.